

Enterprise HR System

Architecture, Implementation, & Deployment Guide



Overview: This document outlines a comprehensive mono-repo structure containing both the React frontend and FastAPI backend. It includes a complete setup guide to instantly generate an Admin environment for local evaluation.

Architecture at a Glance



Component	Technology	Purpose
Frontend	React.js, Tailwind CSS	Client-side dashboard and employee portal UI
Backend	FastAPI (Python)	High-performance, asynchronous REST API
	Prisma Client Python	

Component	Technology	Purpose
Database ORM		Type-safe database queries and schema management
Database	SQLite (Dev) / PostgreSQL (Prod)	Relational data storage
Infrastructure	Docker & Docker Compose	Containerized, reproducible environments

Feature Highlights

- **Role-Based Access Control (RBAC):** Strict separation between Admin dashboards and Employee portals using JWT authentication.
- **Onboarding Pipeline:** Public application endpoints with Admin approval workflows that automatically generate secure employee credentials.
- **Smart Attendance Engine:** Real-time clock-in/clock-out tracking mapped to an automated time-window processor.
- **Automated Payroll:** Dynamic gross and net pay calculations based on exact hours worked multiplied by specific employee hourly rates.
- **Leave Management:** PTO request lifecycle with Admin approval/rejection and employee history tracking.
- **Soft-Delete Architecture:** Employees are "deactivated" rather than deleted to preserve historical payroll and audit records.

Quick Start (Running Locally)

To experience this system on your local machine, ensure you have **Node.js**, **Python 3.9+**, and **Git** installed.

1. Clone the Repository

```
git clone https://github.com/yourusername/Smart-HR-Management-System.git
cd Smart-HR-Management-System
```

2. Start the Backend

Open a terminal and navigate to the backend folder:

```
cd smart-hr-backend
python -m venv venv
# On Windows: venv\Scripts\activate
# On Mac/Linux: source venv/bin/activate
pip install -r requirements.txt
npx prisma generate
npx prisma db push
python seed.py
uvicorn main:app --reload
```

3. Start the Frontend

Open a **new** terminal window and navigate to the frontend folder:

```
cd smart-hr-frontend
npm install
npm run dev
```

Test Credentials: Use the Admin account generated by the seed script to log in.

- Email: **admin@smart.hr**
- Password: **admin123**

Model Design Philosophy

HR systems must prioritize strict data integrity and operational transparency. This project is built around three core engines:

- **The Chrono-Math Engine:** Attendance isn't just a boolean "present/absent." Time is dynamically converted to seconds, translated to hours, and piped directly into the payroll multiplier.
- **Single Source of Truth:** The React frontend acts as a "dumb" viewer. All sensitive logic (tax calculations, role verification, hourly rate lookups) occurs strictly on the server-side to prevent client-side manipulation.
- **Schema-First Design:** Using Prisma allows the database structure (Users, Attendance, Leaves, Payroll) to be perfectly mapped to Python types, ensuring massive reduction in runtime errors.

Project Structure

```
Smart-HR-Management-System/
|-- README.md
|-- .gitignore
|-- smart-hr-frontend/
|   |-- src/
|   |   |-- pages/          # Admin and Employee UI components
|   |   |-- api/            # Axios client configurations
|   |   |-- App.jsx         # React Router map
|   |-- package.json
|-- smart-hr-backend/
|   |-- src/
|   |   |-- modules/        # auth, attendance, payroll, onboarding, leave
|   |   |-- db.py           # Prisma client instantiation
|   |-- prisma/
|   |   |-- schema.prisma   # Master database schema
|-- main.py                 # FastAPI root application
|-- seed.py                 # Initial admin generator
|-- requirements.txt
```